



Язык программирования Java

№ Фамилия Имя – Группа

План занятий

Сегодня

- ① Классы и объекты
- ② Методы и конструкторы
- ③ Фундаментальные принципы ООП

Практика

- ① Закрепление материала с лекции

Java – объектно-ориентированный язык

Каждый объект определяется общим шаблоном – классом

В рамках класса задается общая структура, на основе которой затем создаются объекты. Данные относящиеся к классу, называются **полями** класса, а код для их обработки – **методами**

Объектно-ориентированные программы

- ★ Хорошо структурированы, что повышает их читабельность
- ★ Легко редактировать и тестировать
- ★ Легко дорабатываются и расширяются

Инкапсуляция

Объединение данных и кода для обработки этих данных (методов) в одно целое. Объект является именно той конструкцией, через которую реализуется механизм инкапсуляции

Создание классов

```
1  class MyClass {
2      // Поля
3      String name;
4      int number;
5      // Методы
6      void set(String txt, int num) {
7          name = txt;
8          number = num;
9      }
10     void show() {
11         System.out.println(name + ": " + number);
12     }
13 }
```

Создание объектов

```
1  class Main {  
2      public static void main(String[] args) {  
3          MyClass objA;  
4          objA = new MyClass();  
5          MyClass objB = new MyClass();  
6          objA.set("Объект A", 256);  
7          objB.name = "Объект B";  
8          objB.number = 512;  
9          objA.show(); // Объект A: 256  
10         objB.show(); // Объект B: 512  
11     }  
12 }
```

Закрытые члены класса

```
1 class MyClass {  
2     private String name;  
3     private int number;  
4     void set(String txt, int num) {  
5         name = txt;  
6         number = num;  
7     }  
8     void show() {  
9         System.out.println(name + ": " + number);  
10    }  
11 }  
12 class Main {  
13     public static void main(String[] args) {  
14         MyClass obj = new MyClass();  
15         System.out.println(obj.number);  
16     }  
17 }
```



Ошибка компиляции

The field MyClass.number is not visible

Закрытые члены класса

```
1  class MyClass {
2      private String name;
3      private int number;
4      void set(String txt, int num) {
5          name = txt;
6          number = num;
7      }
8      void show() {
9          System.out.println(name + ": " + number);
10     }
11 }
12 class Main {
13     public static void main(String[] args) {
14         MyClass obj = new MyClass();
15         obj.show();
16     }
17 }
```

Статические поля и методы

Инициализируются с помощью ключевого слова **static** и отличаются от нестатических тем, что они **общие для всех объектов данного класса**

Статические поля и методы

```
1 class MyClass {
2     static int number = 100;
3     void show() {
4         System.out.println(number);
5     }
6 }
7 class Main {
8     public static void main(String[] args) {
9         MyClass objA = new MyClass();
10        MyClass objB = new MyClass();
11
12        MyClass.number = 128;
13        objA.show();
14        objB.show();
15    }
16 }
```

Статические поля и методы

```
static int number = 128;
```

objA

128

objB

128

final

```
1 class MyClass {  
2     final int number = 32;  
3 }  
4 class Main {  
5     public static void main(String[] args) {  
6         MyClass obj = new MyClass();  
7         obj.number = 64;  
8     }  
9 }
```



Ошибка компиляции

The final field MyClass.number cannot be assigned

Анонимный объект

```
1  class MyClass {  
2      void show(String message) {  
3          System.out.println(message);  
4      }  
5  }  
6  class Main {  
7      public static void main(String[] args) {  
8          new MyClass().show("Анонимный объект");  
9      }  
10 }
```

Стандартная ссылка на объект

Нередко в процессе описания методов необходимо явно сослаться на объект, из которого вызывается метод. Проблема в том, что **когда описывается метод, объекта еще не существует**. В таких случаях используется **this**

this

```
1 class MyClass {  
2     String name;  
3     int number;  
4     void set(String txt, int num) {  
5         name = txt;  
6         number = num;  
7     }  
8 }
```

 Упрощенный синтаксис

this

```
1  class MyClass {
2      String name;
3      int number;
4      void set(String name, int number) {
5          this.name = name;
6          this.number = number;
7      }
8  }
9  class Main {
10     public static void main(String[] args) {
11         MyClass obj = new MyClass();
12         obj.set("Java", 25);
13     }
14 }
```

Перегрузка методов

При перегрузке методов **создается несколько методов с одинаковыми именами, но разным количеством и/или типом аргументов**. При вызове метода решение о том, какую его версию следует использовать, принимается на основе контекста команды вызова

Перегрузка методов

```
1 class MyClass {
2     private int first;
3     private int second;
4
5     void set(int a, int b) {
6         first = a;
7         second = b;
8     }
9     void set(int n) {
10        first = n;
11        second = n;
12    }
13    void set() {
14        first = 128;
15        second = 256;
16    }
17 }
18 class Main {
19     public static void main(String[] args) {
20         MyClass obj = new MyClass();
21         obj.set(10, 20); // first: 10, second: 20
22         obj.set(30); // first: 30, second: 30
23         obj.set(); // first: 128, second: 256
24     }
25 }
```

Конструктор

Метод, который вызывается автоматически при создании объекта

Отличия конструктора от обычного метода

- ★ Имя конструктора совпадает с названием класса
- ★ Конструктор не возвращает результат
- ★ Конструктор может иметь аргументы и его можно перегружать

Конструктор

```
1 class MyClass {
2     private int number;
3     private char symbol;
4     void show() {
5         System.out.println("Число: " + number);
6         System.out.println("Символ: " + symbol);
7     }
8     MyClass() {
9         number = 128;
10        symbol = 'J';
11        show();
12    }
13 }
14 class Main {
15     public static void main(String[] args) {
16         MyClass obj;
17         System.out.println("Создается объект");
18         obj = new MyClass();
19     }
20 }
```

Перегрузка конструктора

```
1 class MyClass {
2     private int number;
3     private char symbol;
4     void show() {
5         System.out.println(number + " " + symbol);
6     }
7     MyClass(int number, char symbol) {
8         this.number = number;
9         this.symbol = symbol;
10    }
11    MyClass(int number) {
12        this.number = number;
13        symbol = 'H';
14    }
15    MyClass() {
16        number = 0;
17        symbol = 'W';
18    }
19 }
```

Наследование

Фундаментальный механизм, лежащий в основе любого объектно-ориентированного языка. **Позволяет создавать новые классы на основе уже существующих**

Наследование

Существующий класс, на основе которого создается новый класс, называется **суперклассом**

Класс, который создается на основе **суперкласса**, называется **подклассом**

Наследование

```
final class Tiger
```

```
final class Bird
```

```
class CatFamily
```

```
class Dog
```

```
class Animal
```

Наследование

```
1 class Animal {
2     void makeSound() {
3         System.out.println("Какой-то звук");
4     }
5 }
6 class CatFamily extends Animal {
7     int numberOfLegs = 4;
8 }
9 final class Cat extends CatFamily {
10     void meow() {
11         System.out.println("Мяу");
12     }
13 }
14 class Main {
15     public static void main(String[] args) {
16         Cat basrsikCat = new Cat();
17         basrsikCat.makeSound();
18         basrsikCat.meow();
19         System.out.println(basrsikCat.numberOfLegs);
20     }
21 }
```

Ссылки

